



深圳大学
Shenzhen University

互联网编程

第3章

线程

深圳大学计算机与软件学院

主要内容

- 多线程与线程池编程技术
- **Executor**并发编程框架
- 使用**Executor**的并发程序设计
- 文件压缩并发编程例子
- 同步块编程技术及例子
- 连接线程编程技术及例子



多线程编程技术的恰当应用

- 优点：网络程序，尤其是I/O受限或CPU受限的网络程序，引入多线程，提升并发性能，提高程序性能。
- 缺点：线程本身的创建、切换、清理等会耗费性能资源，对程序性能产生负面影响。
- 合理的度：线程的数目适中。一旦已生成足够多的线程来使用计算机所有可用的空闲时间，再生成更多线程只会将MIPS和内存浪费在线程管理上。



传统多线程处理方式的缺点

- 传统多线程处理方式：使用`new Thread(...).start()`的方法
- 问题：
 - （1）开销大。对于JVM来说，每次新建线程和销毁线程都会有很大的开销。机器的MIPS和内存浪费在线程管理上。
 - （2）线程缺乏管理。没有一个池来限制线程的数量，如果并发量很高，会创建很多的线程，而且线程之间可能会有相互竞争，这将会过多得占用系统资源，增加系统资源的消耗量。而且线程数量超过系统负荷，容易导致系统不稳定。



java多线程并发编程框架

- Jdk1.5以上，`java.util.concurrent.*`中提供了Executor框架封装和隐藏多线程异步控制细节，使回调更易处理。
- Executor框架（并发编程框架）：是指jdk并发库中与Executor相关的功能类，包括线程池、Executor、Executors、ExecutorService、Future、Callable等。
- 编程人员可以创建多个不同线程，并能按期望的顺序得到需要的答案。



线程池 (Thread Pool)

- 作用：当需要限制在应用程序中同一时刻运行的线程数时，可使用线程池编程。线程池作用就是限制系统中执行线程的数量
- 原理：把并发执行的任务提交给一个线程池（不用为每个并发执行的任务都启动一个新的线程）。一旦池里有空闲的线程，就会分配给任务一个线程执行。在线程池的内部，任务被插入一个阻塞队列（**Blocking Queue**），线程池里的线程会去取这个队列里的任务。当一个新任务插入队列时，一个空闲线程就会成功地从队列中取出任务并且执行它。
- 线程池的基本思想：是一种对象池的思想，开辟一块内存空间，里面存放了众多(未死亡)的线程，池中线程执行调度由池管理器来处理。当有线程任务时，从池中取一个，执行完成后线程对象归池，这样可以避免反复创建线程对象所带来的性能开销，节省了系统的资源。
- 多线程服务器的编程实现常用到线程池。每个通过网络到达服务器的连接都被包装成一个任务并且提交给线程池。线程池的线程会并发的处理连接上的请求。
- Java5以上支持线程池编程

线程池方式的优点

- **复用线程。**通过复用创建的了的线程，减少了线程的创建、消亡的开销，每个工作线程都可以被重复利用，可执行多个任务。
- 有效控制并发线程数：可以根据系统的承受能力，调整线程池中工作线线程的数目，防止因为消耗过多的内存，而把服务器累趴下(每个线程需要大约1MB内存，线程开的越多，消耗的内存也就越大，最后死机)。
- 提供了更简单灵活的线程管理。可以提供定时执行、定期执行、单线程、可变线程数等多种线程使用功能。

Executor框架

➤ Executor框架类之间的关系

➤ 在Executor框架中，使用执行器(Executor)来管理Thread对象，从而简化了并发编程

(2)Future接口用来对具体的Runnable或者Callable任务的执行结果进行取消、查询是否完成、获取结果。必要时可以通过get()方法获取执行结果，get()方法会阻塞直到任务返回结果。

(6)CompletionService和

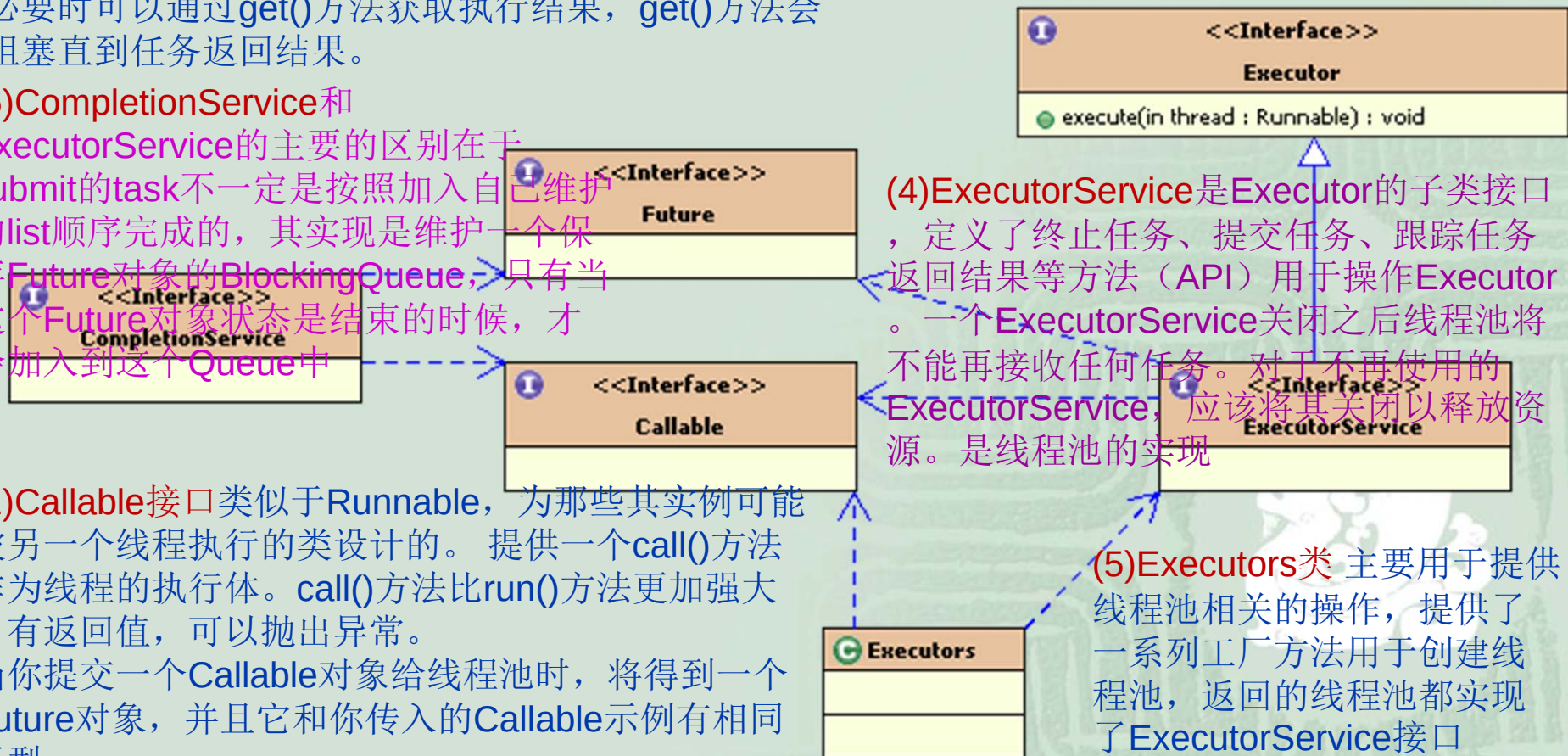
ExecutorService的主要区别在于submit的task不一定是按照加入自己维护的list顺序完成的，其实现是维护一个保存Future对象的BlockingQueue，只有当这个Future对象状态是结束的时候，才会加入到这个Queue中

(1)Callable接口类似于Runnable，为那些其实例可能被另一个线程执行的类设计的。提供一个call()方法作为线程的执行体。call()方法比run()方法更加强大：有返回值，可以抛出异常。当你提交一个Callable对象给线程池时，将得到一个Future对象，并且它和你传入的Callable示例有相同泛型

(3)Executor接口（执行器）将任务的提交与任务的执行分离，定义了一个接收Runnable对象的方法execute。是Executor框架中最基础的一个接口，类似于集合中的Collection接口。

(4)ExecutorService是Executor的子类接口，定义了终止任务、提交任务、跟踪任务返回结果等方法（API）用于操作Executor。一个ExecutorService关闭之后线程池将不能再接收任何任务。对于不再使用的ExecutorService，应该将其关闭以释放资源。是线程池的实现

(5)Executors类 主要用于提供线程池相关的操作，提供了一系列工厂方法用于创建线程池，返回的线程池都实现了ExecutorService接口



异步计算程序结构设计模型

- 多线程编程时，主线程要想获得工作线程（异步计算线程）的结果是比较麻烦的，需要设计特殊的程序结构，比较繁琐而且容易出错。使用**Executor**框架中的**Future**来跟踪异步计算的结果，设计良好的异步计算程序结构模型。
- 根据分而治之的思想，把异步计算的线程按照职责分为3类：
- (1)异步计算的发起线程（控制线程）：负责异步计算任务的分解和发起，把分解好的任务交给异步计算的**work**线程去执行，发起异步计算后，发起线程可以获得**Future**的集合，从而可以跟踪异步计算结果。
- (2)异步计算**work**线程：负责具体的计算任务
- (3)异步计算结果收集线程：从发起线程那里获得**Future**的集合，并负责监控**Future**的状态，根据**Future**的状态来处理异步计算的结果。

控制线程

work线程

计算结果
收集线程

异步计算程序结构设例子

- 应用示例：使用**Executor**框架多线程编程查找一个很大的数字数组中的最大值。P72-73
- Ch3.FindMaxInteger. FindMaxTask.java
- Ch3.FindMaxInteger. MultithreadedMaxFinder.java
- ch3.FindMaxInteger. StartClass.java
- 主要使用**Future**、**Callable**和**Executor**类



FindMaxTask.java

```
FindMaxTask.java ✕
1 package ch3.FindMaxInteger;
2
3 import java.util.concurrent.Callable;
4 /*计算线程：查找数组的一个分段中的最大值*/
5 class FindMaxTask implements Callable<Integer> {
6
7     private int[] data;
8     private int start;
9     private int end;
10
11 FindMaxTask(int[] data, int start, int end) {
12     this.data = data;
13     this.start = start;
14     this.end = end;
15 }
16
17 public Integer call() {
18     int max = Integer.MIN_VALUE;
19     for (int i = start; i < end; i++) {
20         if (data[i] > max) max = data[i];
21     }
22     return max;
23 }
24 }
```



MultithreadedMaxFinder.java

MultithreadedMaxFinder.java

```
3 import java.util.concurrent.*;
4
5 /*控制和计算结果收集线程：分解和发起任务，获得future的集合，并处理计算结果*/
6 public class MultithreadedMaxFinder {
7
8     public static int max(int[] data) throws InterruptedException, ExecutionException {
9
10         if (data.length == 1) {
11             return data[0];
12         } else if (data.length == 0) {
13             throw new IllegalArgumentException();
14         }
15
16         // split the job into 2 pieces
17         FindMaxTask task1 = new FindMaxTask(data, 0, data.length/2);
18         FindMaxTask task2 = new FindMaxTask(data, data.length/2, data.length);
19
20         // spawn 2 threads
21         ExecutorService service = Executors.newFixedThreadPool(2);
22
23         Future<Integer> future1 = service.submit(task1);
24         Future<Integer> future2 = service.submit(task2);
25
26         return Math.max(future1.get(), future2.get());
27     }
28 }
```


StartClass.java

```
StartClass.java ✕
1 package ch3.FindMaxInteger;
2
3 public class StartClass {
4
5     public static void main(String[] args) {
6         // TODO Auto-generated method stub
7
8         int[] num = new int[1000];
9         //生成任意1000整数,0-999之间
10        for(int i=0; i<num.length;i++){
11            num[i] = (int)(Math.random()*1000);
12        }
13        //输出数组中的数
14        for(int j=0; j<num.length;j++){
15            System.out.println(num[j]);
16        }
17        try{
18            int maxNum = MultithreadedMaxFinder.max(num);
19            System.out.println("找出的最大数是: " + maxNum);
20        }catch(Exception ex){
21            System.err.println(ex);
22        }
23
24    }
25
26 }
```

编程应用问题场景:压缩文件

- 对目录中的每一个文件完成gzip压缩。
- 应用特点：大量I/O操作
- 应用特点：数据压缩是CPU密集度高的操作
- 不要同时运行太多线程
- 使用线程池编程



使用线程池实现压缩文件编程

演示运行示例代码:

Ch3.threadPool.GZipAllFiles.java,
GZipRunnable.java

讲析程序结构

➤ 程序设计思想

主程序:
列出目录中的
文件, 确定要
压缩哪个文件

执行速度快

压缩线程:
压缩文件

执行速度慢
个数受限制

ExecutorService
线程池

使主程序与压缩线程
并行运行

运行参数:

C:\temp\ch3data\zip\2018.4.5.pptx

C:\temp\ch3data\zip\build.xml

C:\temp\ch3data\zip\新建 Microsoft
Word 文档 (2).docx

C:\temp\ch3data\zip\新建文本文档.txt

GZipAllFiles.java

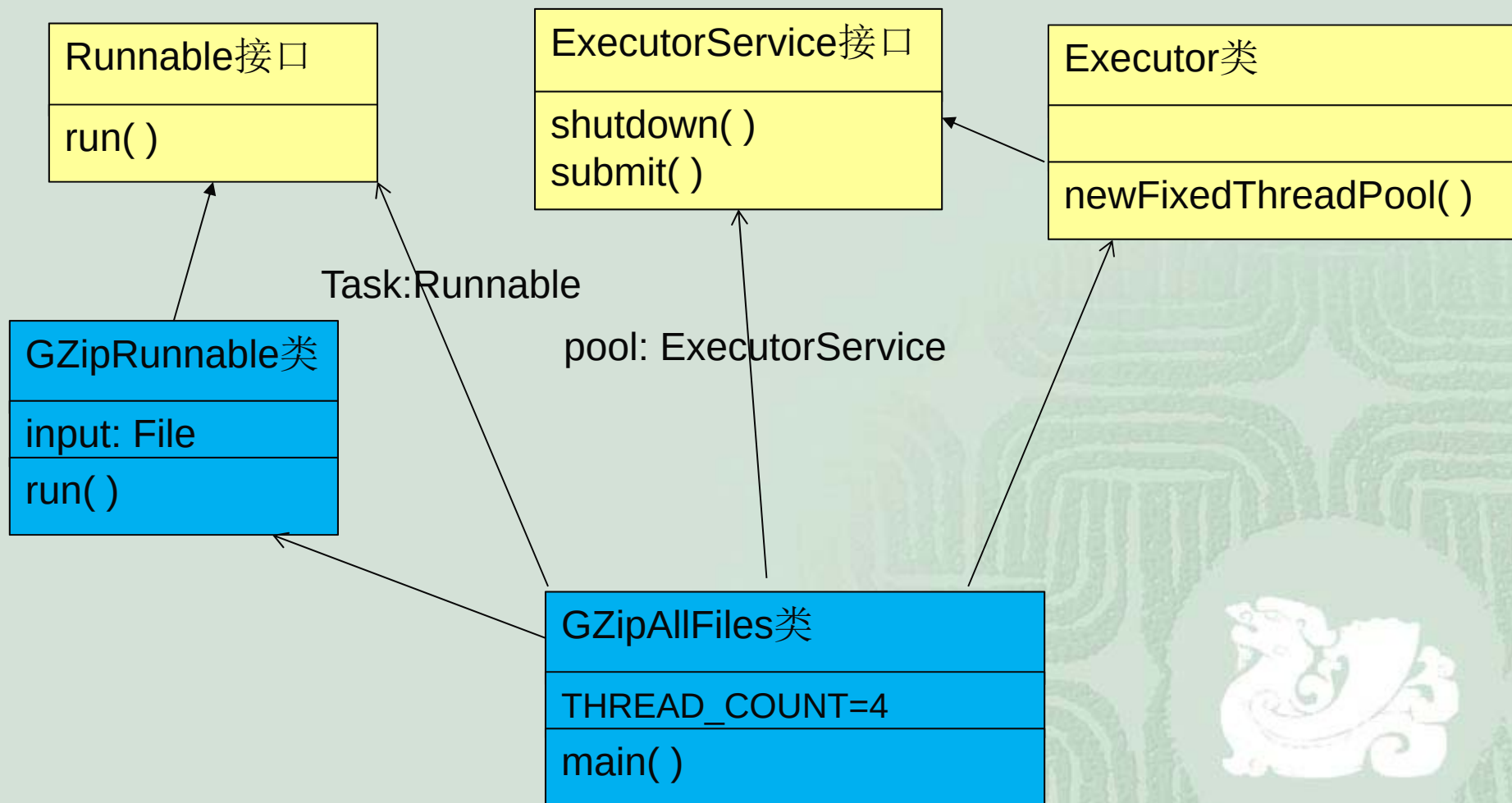
```
GZipAllFiles.java ✖
2 import java.io.*;
3 import java.util.concurrent.*;
4 /*主程序，构造线程池，线程数固定为4，迭代处理参数传入需压缩处理的所有文件和目录。构建工作线程实例，提交到线程池。*/
5 public class GZipAllFiles {
6     public final static int THREAD_COUNT = 4;
7     public static void main(String[] args) {
8         ExecutorService pool = Executors.newFixedThreadPool(THREAD_COUNT);
9         for (String filename : args) {
10             File f = new File(filename);
11             if (f.exists()) {
12                 if (f.isDirectory()) {
13                     File[] files = f.listFiles();
14                     for (int i = 0; i < files.length; i++) {
15                         if (!files[i].isDirectory()) { // don't recurse directories
16                             Runnable task = new GZipRunnable(files[i]);
17                             pool.submit(task);
18                         }
19                     }
20                 } else {
21                     Runnable task = new GZipRunnable(f);
22                     pool.submit(task);
23                 }
24             }
25         }
26         pool.shutdown(); //不会中止等待的工作，只是通知线程池没有新的工作线程任务需要增加到等待队列，池中所有任务完成即可关闭。
27     }
28 }
```



```
1 package ch3.threadPool;
2 import java.io.*;
3 import java.util.zip.*;
4 public class GZipRunnable implements Runnable {
5     private final File input;
6     public GZipRunnable(File input) {
7         this.input = input;
8     }
9     @Override
10    public void run() {
11        // don't compress an already compressed file
12        if (!input.getName().endsWith(".gz")) {
13            File output = new File(input.getParent(), input.getName() + ".gz");
14            if (!output.exists()) { // Don't overwrite an existing file
15                try ( // with resources; requires Java 7
16                    InputStream in = new BufferedInputStream(new FileInputStream(input));
17                    //GZIPOutputStream是一个过滤器流，会压缩它写的所有数据，负责对当前目录中的每一个文件完成GZIP压缩。
18                    OutputStream out = new BufferedOutputStream(
19                        new GZIPOutputStream(
20                            new FileOutputStream(output)));
21                ) {
22                    int b;
23                    while ((b = in.read()) != -1) out.write(b);
24                    out.flush();
25                    //jdk7以后，输入和输出流会在try块的末尾自动关闭。
26                } catch (IOException ex) {
27                    System.err.println(ex);
28                }
29            }
30        }
31    }
32 }
```

使用线程池实现压缩文件编程

➤ 程序结构



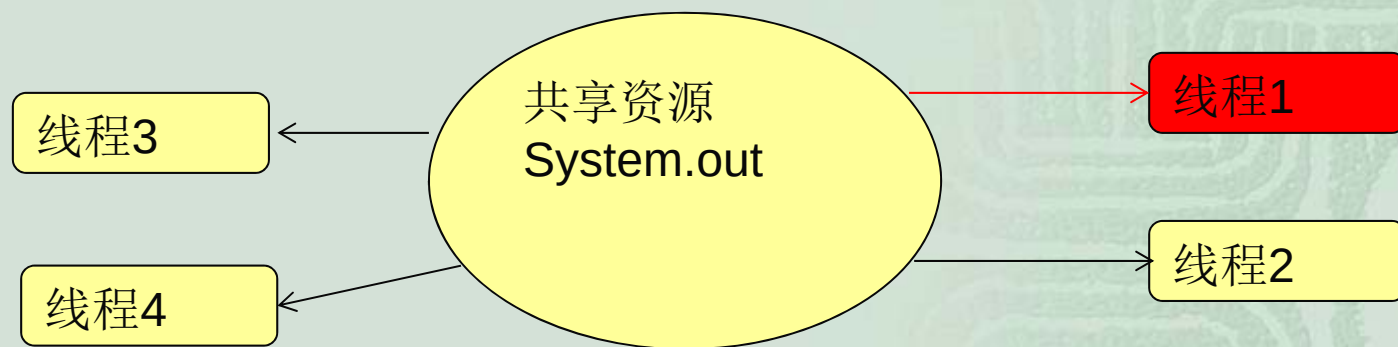
思考拓展：

- Java实现线程池有哪些类？
- Java实现线程池使用了工厂模式，什么是工厂模式？



多线程访问同一资源问题

- 多线程共享的资源有内存、文件句柄、socket等
- 多线程共享资源，若不同时使用共享的资源则可以提高程序效率。
- 若同时使用共享的资源，必须互斥共享。同一时刻只允许一个线程使用，其它线程等待，否则资源会被破坏，程序可能会出现混乱结果。



某一时刻，线程1访问，线程2、3、4等待。

编程时：必须能够指定一个共享资源（若干条语句）只能由一个线程独占访问。

同步块编程技术

- 将需要同步的对象和需要独占访问的语句用 **synchronized** 块围起来。以作同步指示。

需独占访问的语句：

```
System.out.print( input + ":" );
```

```
System.out.print(DatatypeConverter.printHexBinary(digest));
```

```
System.out.println( );
```

加上同步块：

```
Synchronized ( System.out) {
```

```
    System.out.print( input + ":" );
```

```
    System.out.print(DatatypeConverter.printHexBinary(digest));
```

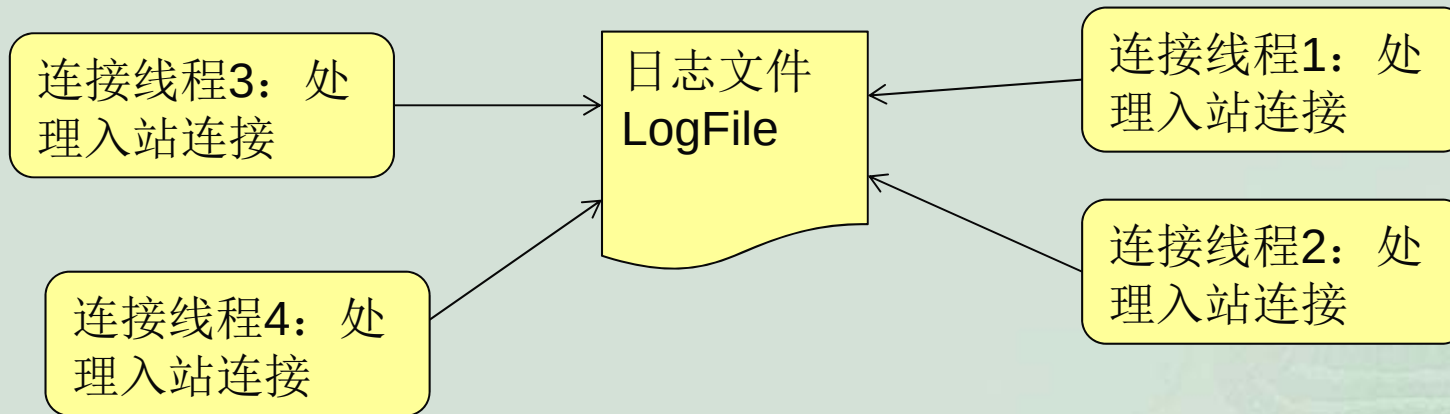
```
    System.out.println( );
```

```
}
```



同步块编程技术应用例子

➤ Web服务器上日志文件的多线程访问



➤ 多个线程会使用LogFile对象的引用获得当前日期和时间，写入到日志文件中。如何编程处理，避免写入混乱？

演示示例代码 **ch3-LogFile.java**
未进行同步处理
三种同步处理方式



LogFile.java

```
LogFile.java ✕
2 import java.io.*;
3 import java.util.*;
4 public class LogFile {
5     private Writer out;
6     public LogFile(File f) throws IOException {
7         FileWriter fw = new FileWriter(f);
8         this.out = new BufferedWriter(fw);
9     }
10    //无同步处理
11    public void writeEntry(String message) throws IOException {
12        Date d = new Date();
13        out.write(d.toString());
14        out.write('\t');
15        out.write(message);
16        out.write("\r\n");
17    }
```



```
18
19- /*同步处理方法一： 选择对Writer对象out同步
20  *
21  * public void writeEntry(String message) throws IOException {
22  synchronized (out){
23      Date d = new Date();
24      out.write(d.toString());
25      out.write('\t');
26      out.write(message);
27      out.write("\r\n");
28  }
29  }
30  * */
31
32- /*同步处理方法二： 选择对LogFile对象本身同步
33  *
34  * public void writeEntry(String message) throws IOException {
35  synchronized (this){
36      Date d = new Date();
37      out.write(d.toString());
38      out.write('\t');
39      out.write(message);
40      out.write("\r\n");
41  }
42  }
```

LogFile.java

LogFile.java

```
43     * */
44     /*同步处理方法三：选择使用synchronized修饰符对对象本身同步整个方法体
45     *
46     * public synchronized void writeEntry(String message) throws IOException {
47         Date d = new Date();
48         out.write(d.toString());
49         out.write('\t');
50         out.write(message);
51         out.write("\r\n");
52     }
53     * */
54     public void close() throws IOException {
55         out.flush();
56         out.close();
57     }
58 }
59
```

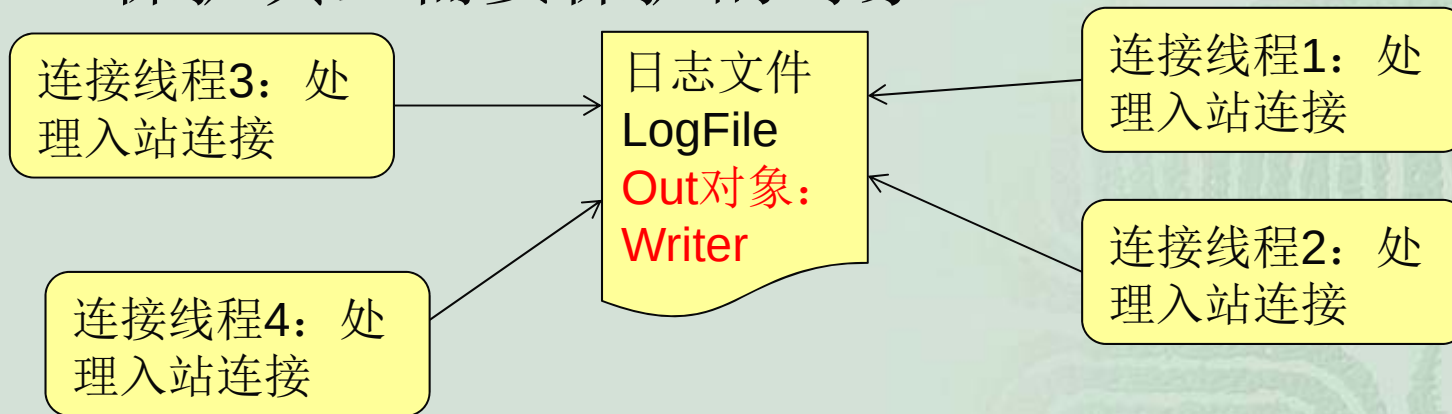


编程思考问题？

- 改进上次编程思考问题中完成的文件安全算法摘要保存程序，给出文件安全算法摘要日志生成程序。要求：使用多线程同步编程技术，将文件安全算法摘要计算程序（使用多线程计算）算出的文件安全算法摘要保存到一个日志文件中。例如，对于1.txt文件计算生成的安全算法摘要存入名为SafeLogFile.txt的日志文件中，对于2.xml文件计算生成的安全算法摘要也存入名为SafeLogFile.txt文件中。并且在向SafeLogFile.txt日志文件中存入某个文件的安全算法摘要时，要一并存入该文件安全算法摘要的生成时间。

同步块编程技术可能引起的问题

- 1.可能使VM的性能严重下降,代码执行速度降低三分之一或更多
- 2.大大增加了死锁的可能性
- 3.并不是对象本身需要防止同时修改或访问,如果只是对该方法所属类的实例进行同步,可能并不能保护真正需要保护的对象.



造成的问题: 目标是为了避免4个线程同时使用out对象写入日志文件,但如果其他类有与LogFile完全无关的out的引用,这些写入会失败。

同步编程技术的替换技术

- 使用同步编程技术解决因线程调度可能引起不一致问题，又会引入新的问题，称为同步问题。
- 思考：有哪些替换编程技术，可以解决多线程资源共享可能造成的不一致问题，又能避免使用同步而引起同步问题？
- 方法1：尽量使用局部变量而不是字段
- 方法2：基本类型方法参数，构造函数通常都是线程安全的
- 方法3：将非线程安全类用作线程安全的类的一个私有字段。



线程调度

- 多线程运行程序要考虑线程调度问题
- 重要的线程能够得到CPU时间来运行，更重要的线程能够得到更多的时间,现在能够以合理的顺序执行
- 优先级：在Java中，每个线程都有一个优先级，指定为一个从0~10的整数.虚拟机优先调度优先级高的线程执行. 10是最高优先级，0是最低，默认优先级为5，通常我们编写程序中的线程都是默认优先级.
- Thread.MIN_PRIORITY = 1, Thread.NORM_PRIORITY = 5, Thread.MAX_PRIORITY = 10



线程调度

- 线程调度器:每个虚拟机都有一个线程调度器, 确定在给定时刻运行哪个线程
- 线程调度方式:抢占式和协作式
- 抢占式线程调度器定期强行中断当前正在执行的线程, 将CPU控制权交给另外的线程。
- 协作式线程调度器在将CPU控制权交给其他线程前会等待正在运行的线程自己暂停。
- 在编写Java线程程序实现run（）方法时, 要确保线程有可以暂停的条件, 让其他线程有机会运行。
- 可以使一个线程暂停的方式有:
- 对I/O阻塞、对同步对象阻塞、放弃、休眠、连接另一个线程、等待一个对象、结束、可以被更高优先级线程抢占 等



思考问题

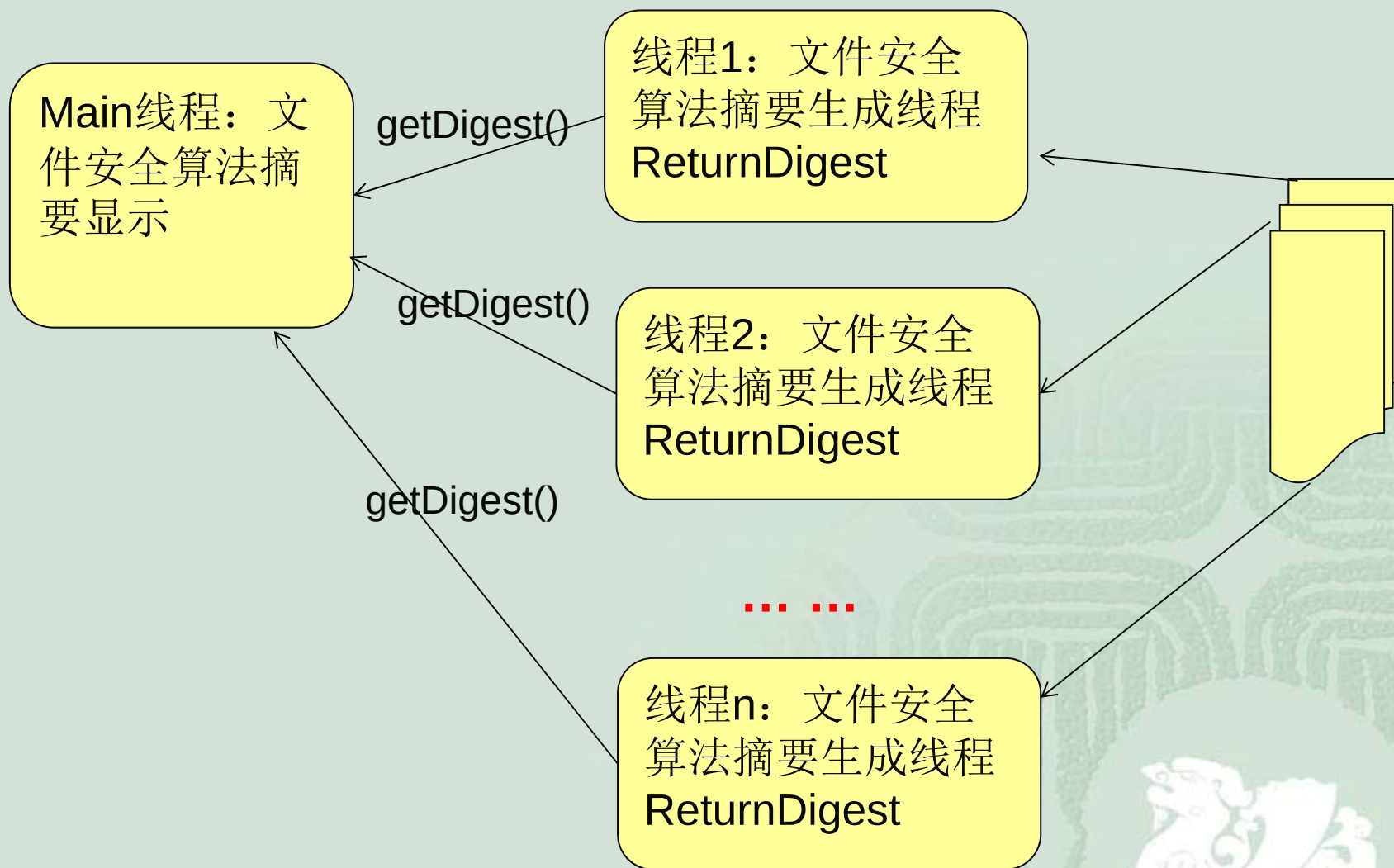
- 1.什么是线程死锁问题
- 2.什么是线程饥饿问题

线程死锁：如果两个线程需要独占访问同样的一个资源集，而每个线程分别有这些资源的不同子集的锁，就会发生死锁。

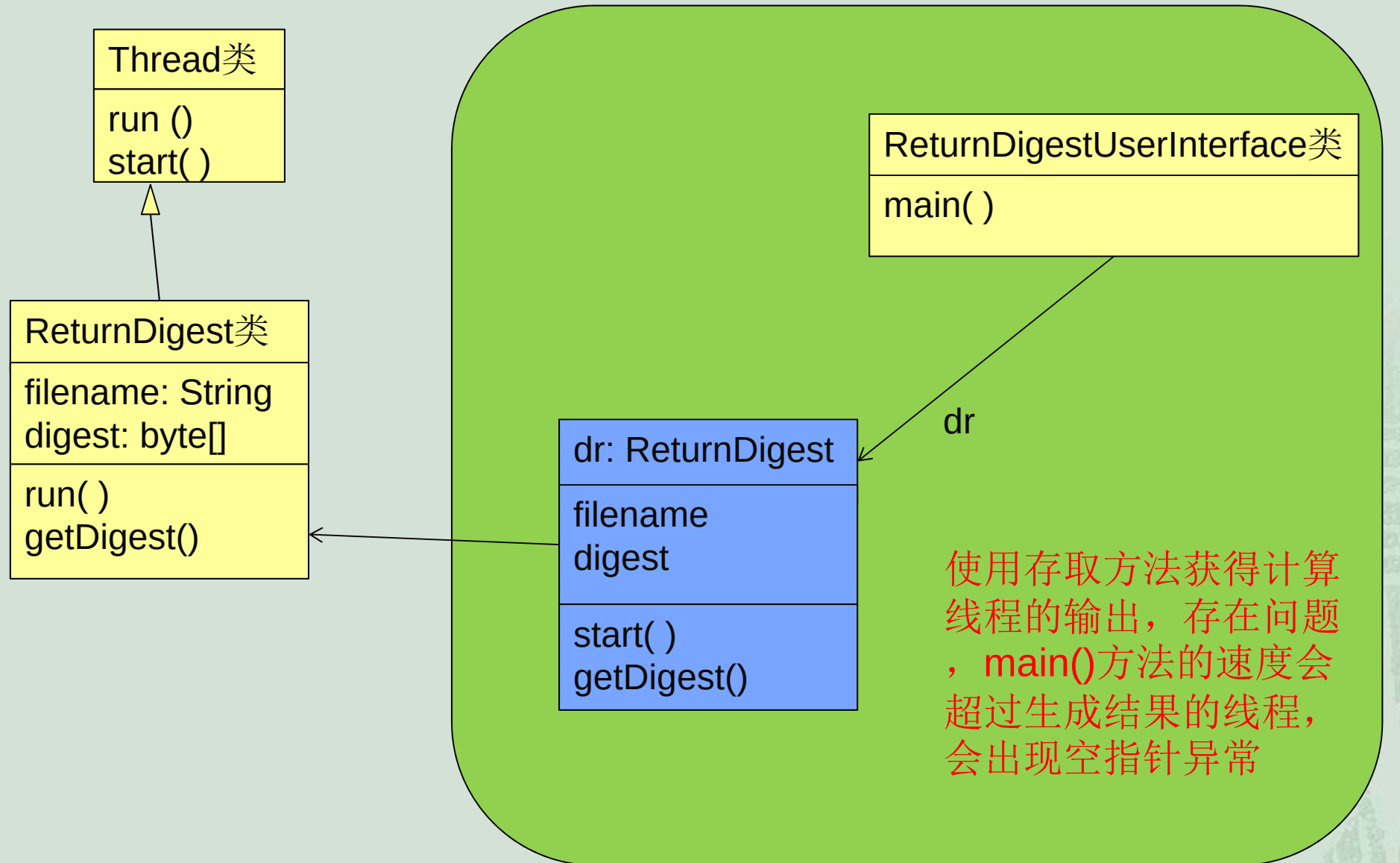
线程饥饿：在线程调度运行时，有的线程一直得不到CPU时间片，一直处于等待状态，无法运行任务。



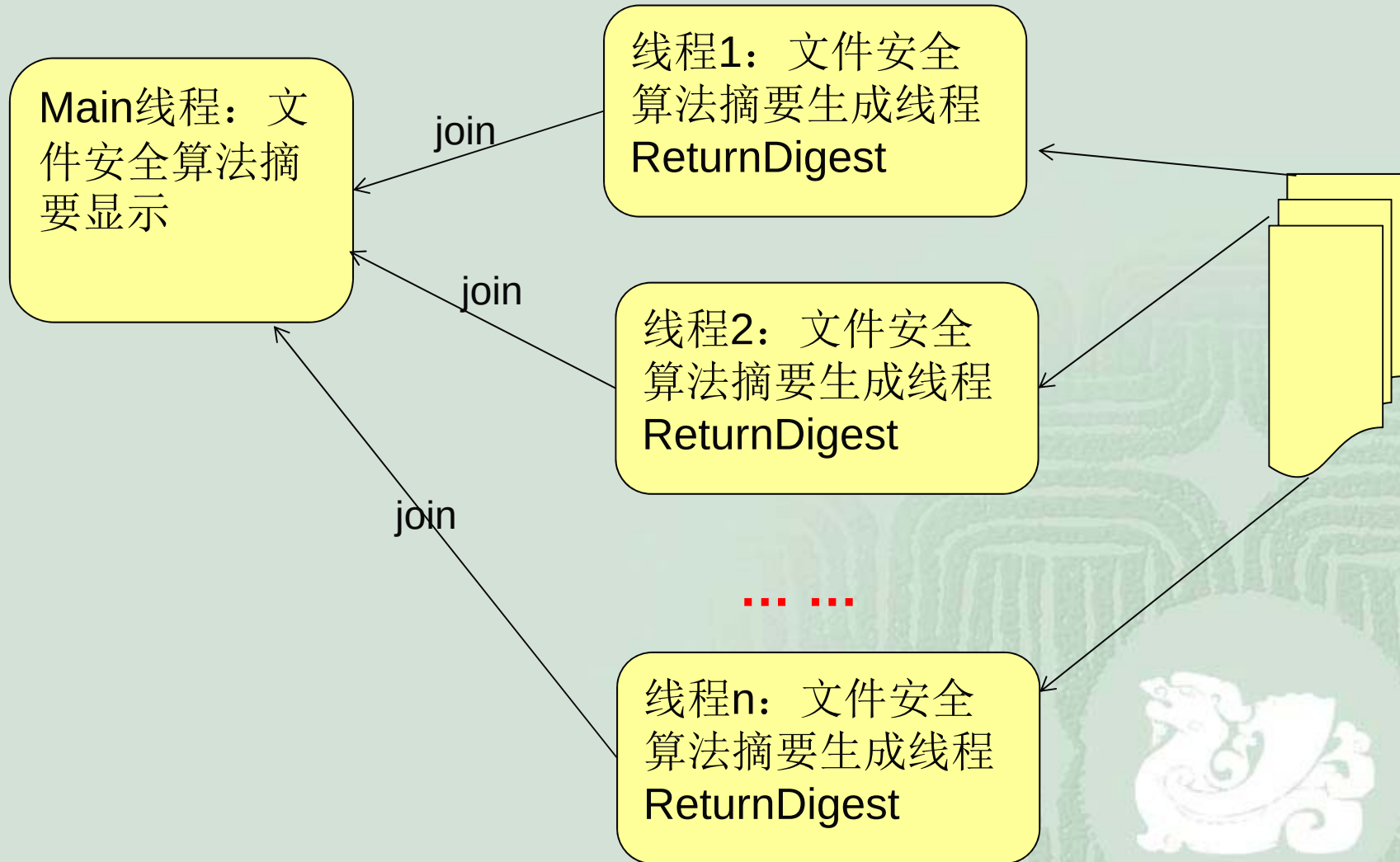
编程问题场景： 计算多个文件安全算法摘要并显示



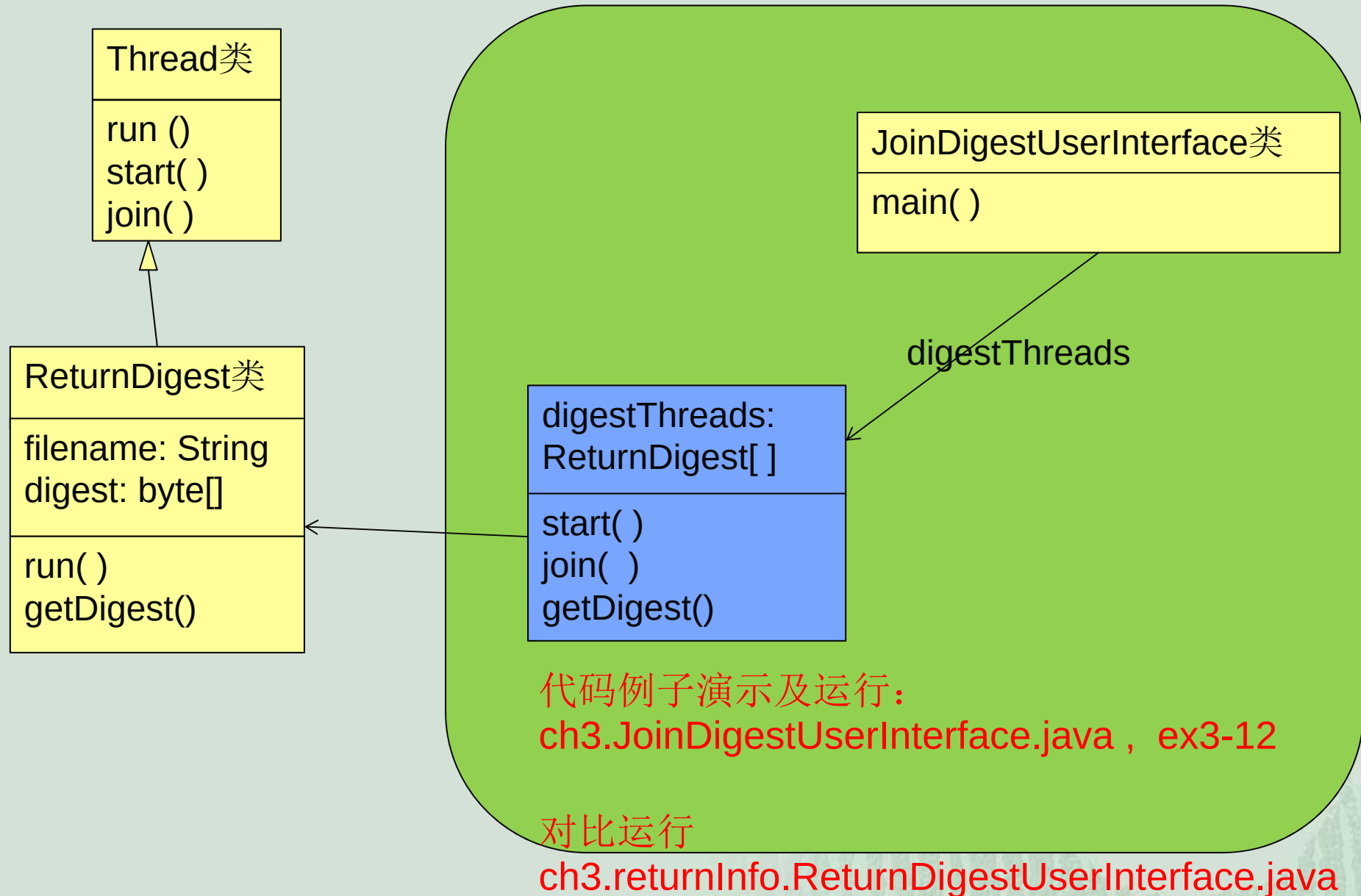
问题续：使用存取方法的多线程程序设计



问题解决：使用连接线程



使用连接线程方法的多线程程序设计



```
2 import javax.xml.bind.DatatypeConverter;
3 import ch3.returnInfo.ReturnDigest;
4 //使用连接生成所需结果的线程，避免竞态条件 ex3-12
5 public class JoinDigestUserInterface {
6
7     public static void main(String[] args) {
8         ReturnDigest[] digestThreads = new ReturnDigest[args.length];
9         for (int i = 0; i < args.length; i++) {
10             // Calculate the digest
11             digestThreads[i] = new ReturnDigest(args[i]);
12             digestThreads[i].start();
13         }
14         for (int i = 0; i < args.length; i++) {
15             try {
16                 digestThreads[i].join();
17                 // Now print the result
18                 StringBuffer result = new StringBuffer(args[i]);
19                 result.append(": ");
20                 byte[] digest = digestThreads[i].getDigest();
21                 result.append(DatatypeConverter.printHexBinary(digest));
22                 System.out.println(result);
23             } catch (InterruptedException ex) {
24                 System.err.println("Thread Interrupted before completion");
25             }
26         }
27     }
28 }
```

连接线程编程技术相关概念

- 通过调用join()方法，允许一个线程在继续执行前等待另一个线程结束。
- 调用join()方法的线程是**连接线程 (join thread)**，等待**被连接的线程(joined thread)**（join方法属于被连接的线程）。
- 思考：上述ex3-12程序设计中，哪个是连接线程？哪个是被连接的线程？



连接线程编程技术相关概念

- 通过调用`join()`方法，允许一个线程在继续执行前**等待**另一个线程结束。连接线程要等待多久？
- 取决于`join()`的参数：
- `join()` ----无限等待被连接的线程结束
- `join( long milliseconds )`---- 等待`milliseconds`微秒，然后继续执行。
- `join( long milliseconds, int nanoseconds)` ---等待`nanoseconds`毫秒或`milliseconds`微秒，然后继续执行



谢谢

